

8. UNIX C Socket

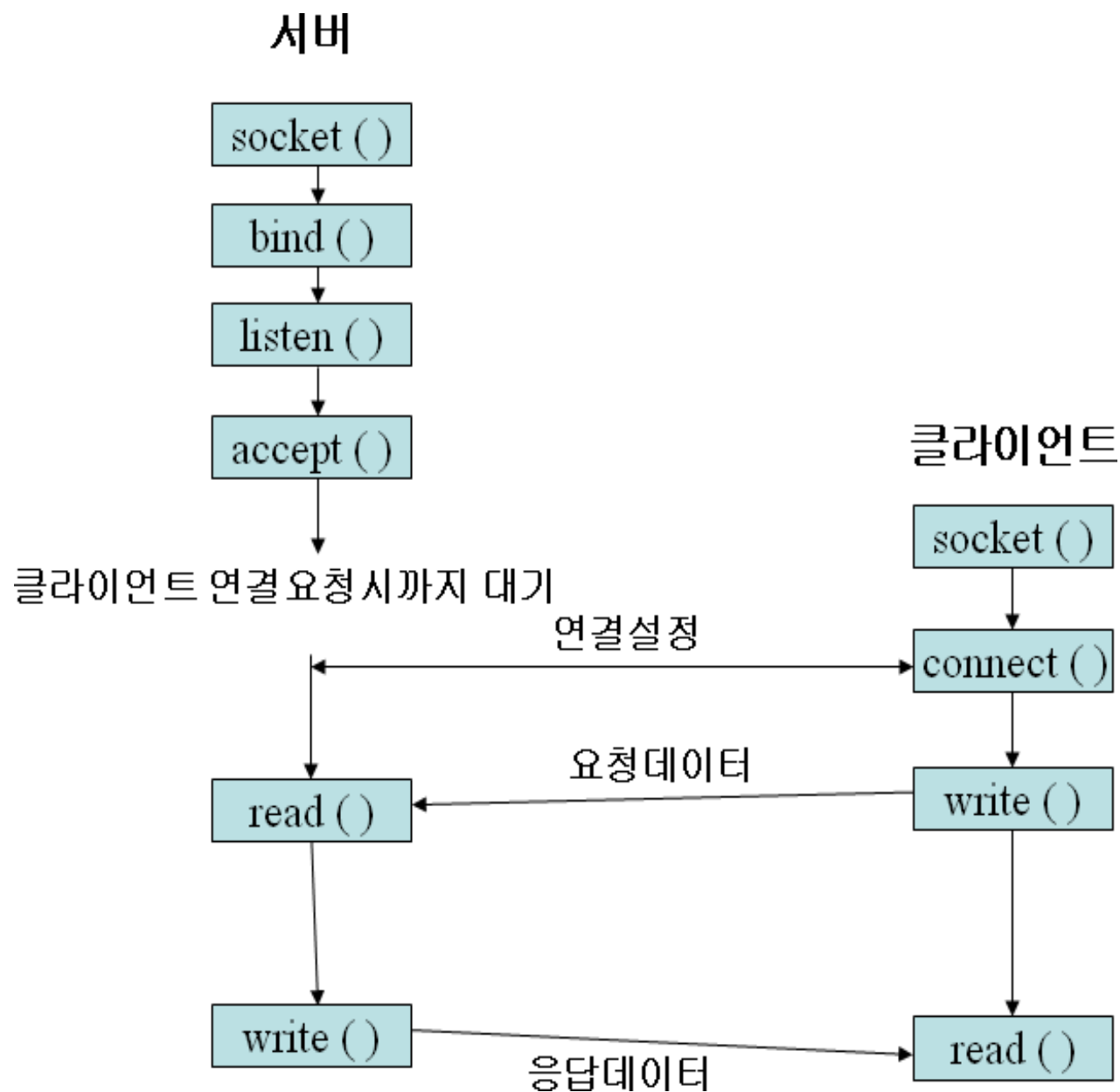
소켓이란

- 응용프로그램을 개발할 때 TCP/UDP 또는 IP(raw socket)를 이용하여 프로그램을 개발 할 수 있도록 지원
- 응용프로그램 개발하기 위해 하위 계층 프로토콜을 사용할 수 있도록 제공
 - TCP: mail, web, ftp, telnet 등 신뢰성 있는 전송이 필요한 경우
 - UDP: DNS, 실시간 스트리밍 서비스, 등 connectionless 전송 이 필요한 경우
 - IP: ping, traceroute 같은 IP 계층을 이용한 프로그램일 경우

소켓이란

- 유닉스/리눅스에서의 소켓 인터페이스
 - `<sys/socket.h>`
 - 파일 입출력(I/O)과 유사한 구조
 - 유닉스/리눅스에서 다음과 같이 파일을 오픈하면
 - `fd = open("sample.txt", O_RDONLY)`
 - 양의 정수값을 리턴하는데 이를 파일 디스크립터(file descriptor, 일종의 포인터)라고 하고 프로그램에서 이 오픈된 파일을 액세스할 때 이 파일 디스크립터를 사용
 - `read(fd, buf, 512)`
- 소켓 디스크립터
 - `sd = socket(AF_INET, SOCK_STREAM, 0)`

소켓을 이용한 네트워크 연결 절차



- 서버 측:

1. `socket()` : 소켓을 생성합니다.
2. `bind()` : 소켓에 IP 주소와 포트 번호를 할당하여 '문패'를 겁니다.
3. `listen()` : 클라이언트의 연결 요청을 들을 준비를 합니다.
4. `accept()` : 클라이언트의 연결 요청을 수락하고, 새로운 소켓을 생성해 클라이언트와 1대1 통신을 시작합니다.
5. `read()` / `write()` : 클라이언트와 데이터를 주고받습니다.
6. `close()` : 통신을 마친 후 소켓을 닫습니다.

- 클라이언트 측:

1. `socket()` : 소켓을 생성합니다.
2. `connect()` : 서버의 IP 주소와 포트 번호로 연결을 시도합니다.
3. `write()` / `read()` : 서버와 데이터를 주고받습니다.
4. `close()` : 통신을 마친 후 소켓을 닫습니다.

- 소켓 생성 함수

int socket(int family, int type, int protocol)

- Family

AF_INET: AF_INET (인터넷 주소 체계) : 주로 사용

AF_UNIX (유닉스 주소 체계)

AF_INET6 (128비트 IPv6 주소 체계) : IPv6일 때 사용

- Type: 서비스 타입(type of service)을 의미

연결형(stream) 서비스를 위해서는 SOCK_STREAM을,

비연결형(datagram) 서비스를 위해서는 SOCK_DGRAM을 사용

- Protocol: 소켓 지원 프로그램 지정, 보통 0 사용

소켓과 주소를 묶어주는 함수 bind

```
int bind(int socket_fd,  
         (struct sockaddr *)&server_addr,  
         sizeof(server_addr)) ;
```

- 외부에서 소켓으로 접속하기 위해서는 생성한 소켓과 사용할 주소를 묶어주는 작업이 서버에서는 필요
- 클라이언트에서 서버의 주소와 포트 정보를 이용하여 연결 접속 시도하기 때문
- 성공시 0 실패 시-1

클라이언트의 연결을 대기하는 listen

```
int listen(int s, int backlog) ;
```

int s : 소켓 번호

int backlog : 연결을 기다릴 수 있는 클라이언트의 최대 수

- 클라이언트에서 이 소켓으로 연결요청(connect())하면
TCP 연결을 설정하고 서버에서 연결을 받아들이기 위해
accept() 수행

클라이언트의 연결을 수락하는 accept()

```
int accept(int s, struct sockaddr *addr,  
           int *addrlen);
```

int s : 소켓 번호

struct sockaddr *addr : 연결 요청을 한 클라이언트의 소켓주소 구조체

int *addrlen : *addr 구조체 크기의 포인터

- 성공하면 클라이언트와 통신할 수 있는 새로운 소켓이 생성하여 반환된다 (실패시 -1)
- 서버는 이 후로는 이 소켓으로 클라이언트와 통신하게 됨

송신 함수

```
int send(int s, char * buf, int length, int flags)
```

int s : 소켓,

char *buf: 전송할 데이터

int length: 데이터 길이

int flags: 0

반환값: 보낸 바이트 수, 음수면 실패

```
int write(int s, const void* buf, int length)
```

int s: 소켓

buf: 전송할 데이터

length: 데이터 길이

반환값: 보낸 바이트 수, 음수면 실패

```
write(client_fd, buffer, msg_size);
```

수신 함수

```
int recv(int s, char * buf, int length, int flags)
```

int s : 소켓,

char *buf: 전송할 데이터

int length: 데이터 길이

int flags: 0

반환값: 받은 바이트 수, 음수면 실패

```
int read(int s, const void* buf, int length)
```

int s: 소켓

buf: 전송할 데이터

length: 데이터 길이

반환값: 받은 바이트 수, 음수면 실패

```
msg_size = read(client_fd, buffer, BUF_LEN);
```

TCP에서 송수신

write() 또는 send() 실행하면 송신

1. TCP의 송신 버퍼로 데이터가 들어감
2. 송신버퍼가 full 이면 write() send() 함수에서 대기
3. 버퍼가 비면 버퍼로 데이터 전송
4. 버퍼에 데이터가 다 들어가면 write() send() 반환

read() 또는 recv() 실행하면 수신 대기

1. 이 문장을 수행하면 소켓을 통해 데이터가 수신 버퍼에 도착할 때까지 대기
2. 수신버퍼에 데이터가 도착하면 수신 반환

* accept() 와 수신 부분에서 대기 발생

종료 함수

int close(int s)

int s : 소켓,

반환값: 보낸 바이트 수, 음수면 실패

서버나 클라이언트 어느 쪽이든 먼저 호출 가능

서버나 클라이언트의 버퍼에 데이터가 있어 전송 중이면 전송을 끝내고 종료

서버로 연결요청하는 함수 connect

```
int connect(int socket,  
            const struct sockaddr *addr, int addrlen)
```

int socket : 소켓 번호(디스크립터)

const struct sockaddr *addr : 상대방 서버의 소켓서버 구조체

int addrlen : 구조체 *addr의 크기

성공 시 0, 실패 시 -1 반환

서버가 `listen()` `accept()` 호출하여 대기하고 있지 않으면 에러 발생

리눅스 명령어 또는 함수 설명을 볼 수 있는 사이트 <https://man7.org/>

구조체와 헤더파일

- `<sys/types.h>` : 데이터 타입 정의
- `<sys/socket.h>` : 소켓 함수 및 관련 구조체
- `<netinet/in.h>` : 인터넷 주소 체계(IPv4, IPv6) 관련 구조체 (`sockaddr_in`)
- `<arpa/inet.h>` : IP 주소 변환 함수 (`inet_addr` , `inet_ntoa`)

```
struct sockaddr_in {  
    sa_family_t    sin_family; // 주소 체계 (AF_INET)  
    in_port_t      sin_port;   // 포트 번호 (네트워크 바이트 순서)  
    struct in_addr  sin_addr;  // IP 주소  
    char           sin_zero[8]; // 사용되지 않음  
};
```

소켓 주소를 담을 구조체

```
struct sockaddr_in server_addr;
```

```
//주소 패밀리 설정
```

```
server_addr.sin_family = AF_INET;
```

```
// IP v4 주소 설정
```

```
#define SERV_IP_ADDR "192.168.0.1"
```

```
server_addr.sin_addr.s_addr = inet_addr(SERV_IP_ADDR);
```

```
// 또 다른 방법
```

```
server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
```

```
// 포트 설정
```

```
server_addr.sin_port = htons(atoi(argv[1]));
```

* in_addr inet_addr(const char *string)
문자열을 32비트 정수 인터넷 주소 형태로
변환하는 함수

* uint32 htonl(uint32)
리틀 엔디안에서 빅 엔디안으로 변환하는 함수
long 형으로
* uint16 htons(uint16)
short 형으로

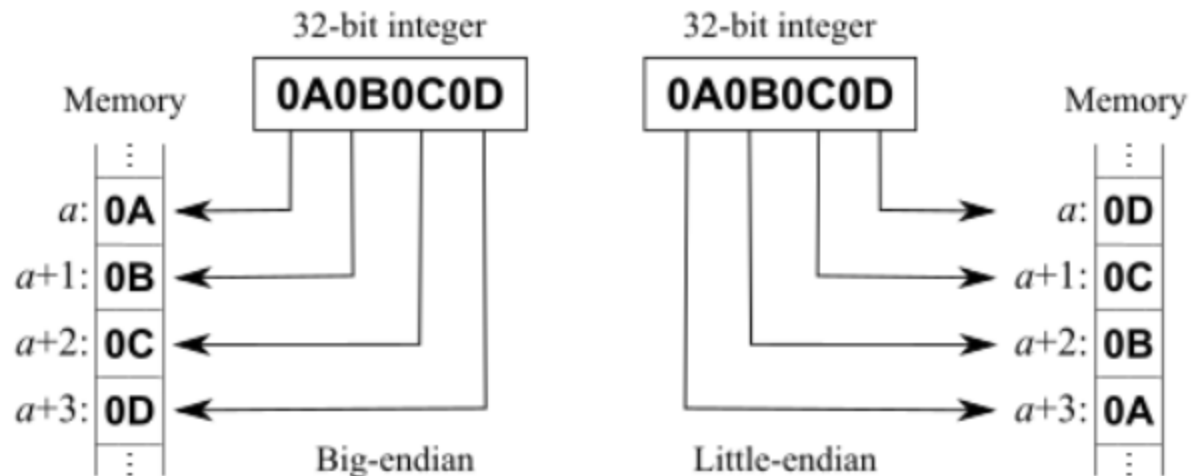
- 1byte = char
- 2byte = short
- 4byte = long

- int atoi(string) 문자열을
정수로 변환하는 함수

빅 엔디안과 리틀 엔디안의 차이

- 네트워크 통신은 빅 엔디안으로 통일하자
- 저수준 통신에서는 반드시 고려해야함

- 메모리에 값을 저장 하는 방식
 - 리틀 엔디안 little endian
 - : 값을 작은 바이트 부분부터 메모리에 저장
 - : 주로 Intelx86과 ARM 아키텍처를 사용하는 PC에서 사용
 - 빅 엔디안 big endian
 - : 큰 부분부터 메모리에 저장
 - : 주로 통신에서 사용



사실은 같은 숫자이나 다른 숫자로 인식하게 됨
 $0A0B0C0D \neq 0D0C0B0A$

echo Server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
```

//소켓 프로그래밍에 사용될 헤더파일 선언

```
#define BUF_LEN 128
//메시지 송수신에 사용될 버퍼 크기를 선언
```

echoServer.c

```
int main(int argc, char *argv[]){
    char buffer[BUF_LEN];
    struct sockaddr_in server_addr, client_addr;
    char temp[20];
    int server_fd, client_fd;
    //server_fd, client_fd : 각 소켓 번호

    int len, msg_size;
    if(argc != 2)    {
        printf("usage : %s [port]\n", argv[0]);
        exit(0);
    }
```

```
if((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == -1)
{
    // 소켓 생성
    printf("Server : Can't open stream socket\n");
    exit(0);
}
memset(&server_addr, 0x00, sizeof(server_addr));
//server_addr 을 NULL로 초기화

server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
server_addr.sin_port = htons(atoi(argv[1]));
//server_addr 셋팅
```

```
if (bind(server_fd,  
        (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0)  
{//bind() 호출  
    printf("Server : Can't bind local address.\n");  
    exit(0);  
}  
if (listen(server_fd, 5) < 0) { //소켓을 수동 대기모드로 설정  
    printf("Server : Can't listening connect.\n");  
    exit(0);  
}
```

```
memset(buffer, 0x00, sizeof(buffer));  
printf("Server : wating connection request.\n");  
len = sizeof(client_addr);  
  
while(1) {  
    client_fd = accept(server_fd,  
                      (struct sockaddr *)&client_addr, &len);  
    if(client_fd < 0) {  
        printf("Server: accept failed.\n"); exit(0);  
    }  
}
```

```
    inet_ntop(AF_INET, &client_addr.sin_addr.s_addr, temp, sizeof(temp));  
    printf("Server : %s client connected.\n", temp);
```

```
    msg_size = read(client_fd, buffer, BUF_LEN);  
    write(client_fd, buffer, msg_size);  
    close(client_fd);  
    printf("Server : %s client closed.\n", temp);
```

```
}
```

```
close(server_fd);
```

```
return 0;
```

```
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>

#define BUF_LEN 128
//메시지 송수신에 사용될
//버퍼 크기를 선언

void main(int argc, char *argv[]){
    int s_sock, n;
    struct sockaddr_in server_addr;
    //struct sockaddr_in server_addr :
    // 서버의 소켓주소 구조체
    char buffer[BUF_LEN];

    if(argc != 3)
    {
        printf("usage : %s [ip_Address]
               [port]\n", argv[0]);
        exit(0);
    }
```



```
bzero((char *)&server_addr,  
      sizeof(server_addr));  
//서버의 소켓주소 구조체 server_addr을 NULL로 초기화
```

```
server_addr.sin_family = AF_INET; //주소 체계를 AF_INET 로 선택  
server_addr.sin_addr.s_addr = inet_addr(argv[1]);  
//32비트의 IP주소로 변환  
server_addr.sin_port = htons(atoi(argv[2]));  
// 서버 포트 번호
```

```
if((s_sock = socket(AF_INET, SOCK_STREAM, 0)) < 0)  
{//소켓 생성과 동시에 소켓 생성 유효검사  
    printf("can't create socket\n");  
    exit(0);    }
```

```
if(connect(s_sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0)
    {//서버로 연결요청
        printf("can't connect.\n");
        exit(0);
    }
memset(buffer, 0x00, sizeof(buffer));
char message[] = "Hello, I am a client";
if (write(s_sock, message, sizeof(message)-1) < 0)
    printf("write error\n");
n = read(s_sock, buffer, BUF_LEN);
printf("Message from server: %s\n",  buffer);
close(s_sock);
//사용이 완료된 소켓을 닫기
}
```

윈도우에서 리눅스 환경 구축하기

- 명령프롬프트에서 `wsl -install` 로 ubuntu 다운로드 및 설치
- 계정 생성: 원하는 계정과 패스워드 설정
 - 예시에서는 npp/kbu1950으로 함

`wsl -install`

Create a default Unix user account:

New password:

Retype new password:

cmd npp@DESKTOP-TQ836S2: /mnt/c/Users/User

Microsoft Windows [Version 10.0.19045.6332]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>wsl --install

다운로드 중: Ubuntu

설치 중: Ubuntu

패치가 설치되었습니다. 'wsl.exe -d Ubuntu'을(를) 통해 시작할 수 있습니다.

Ubuntu 시작하는 중...

Provisioning the new WSL instance Ubuntu

This might take a while...

Create a default Unix user account: root

fatal: The user 'root' already exists.

Failed to create user 'root'. Please choose a different name.

Create a default Unix user account: npp

New password:

Retype new password:

passwd: password updated successfully

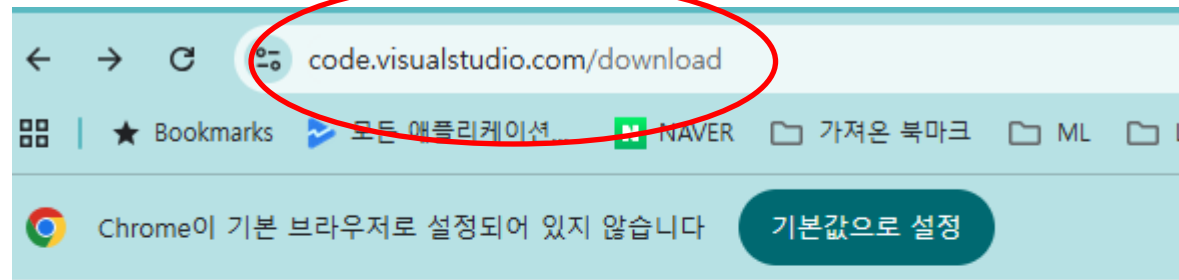
To run a command as administrator (user "root"), use "sudo <command>".

See "man sudo_root" for details.

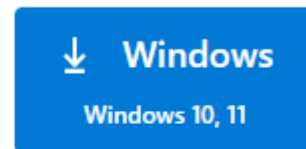
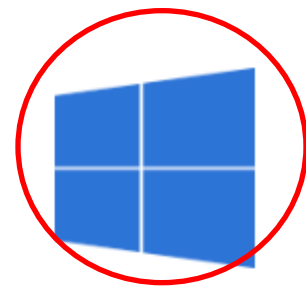
npp@DESKTOP-TQ836S2: /mnt/c/Users/User\$

Visual Studio Code 설치 후 wsl 연동하기

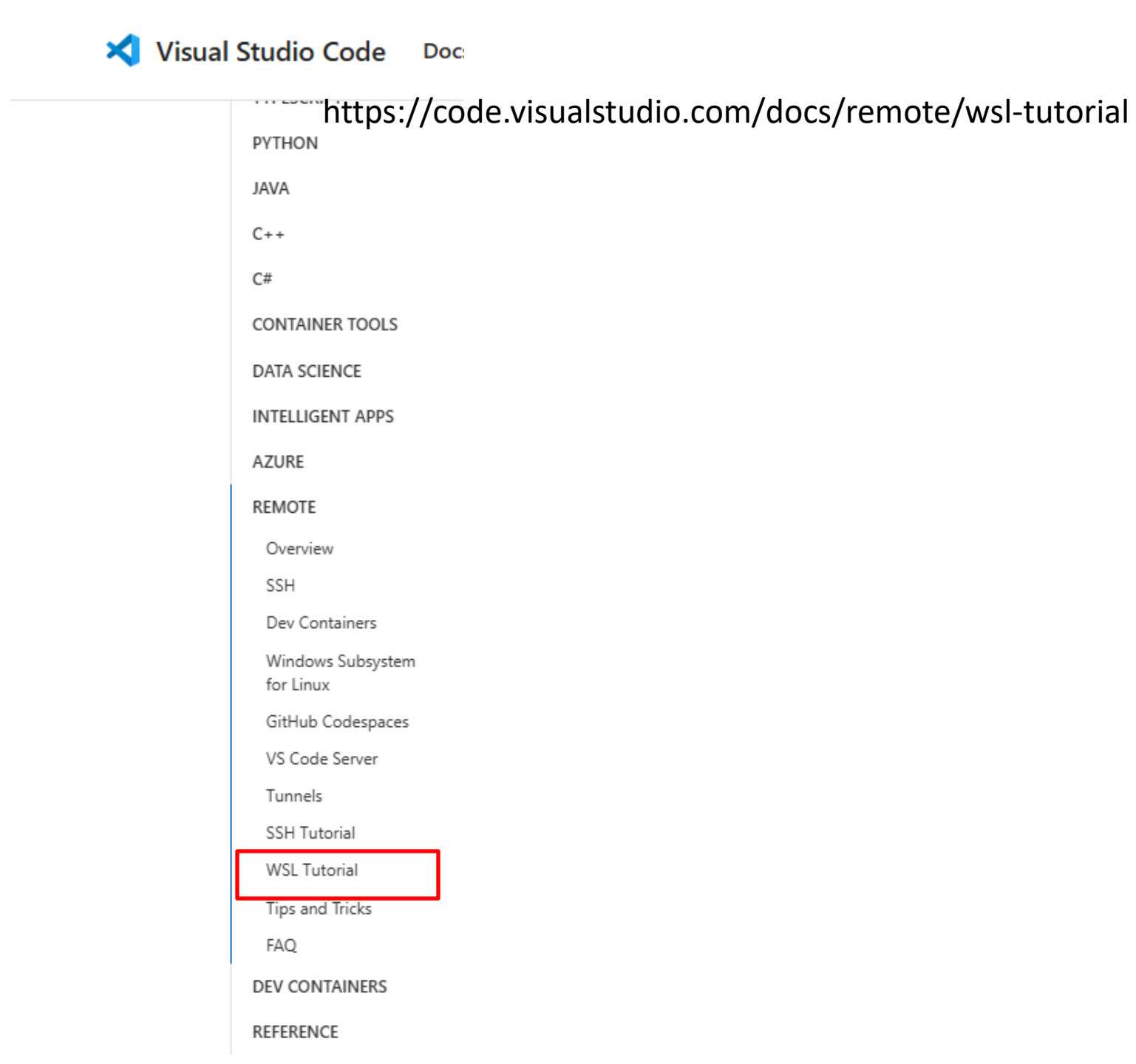
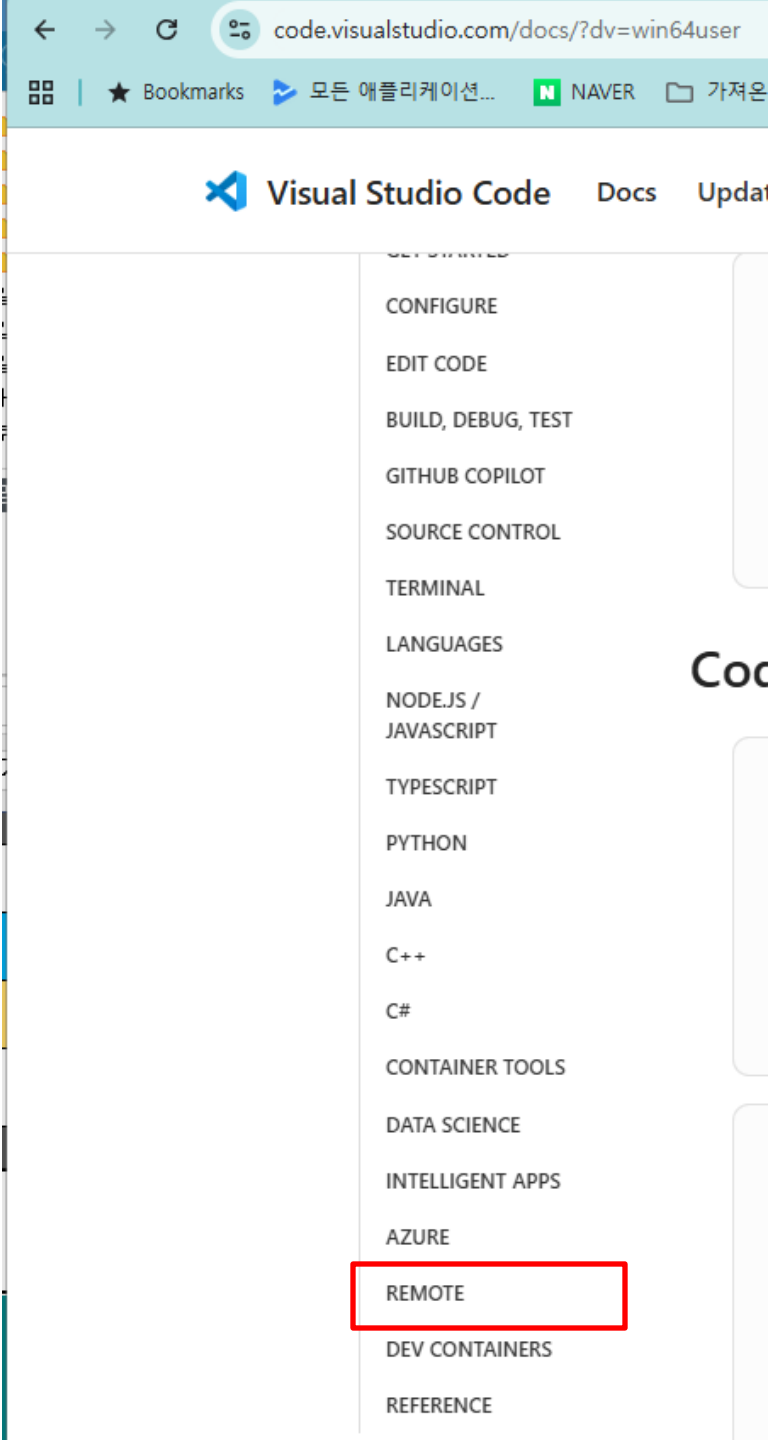
- 최신 Visual Studio Code 설치하기

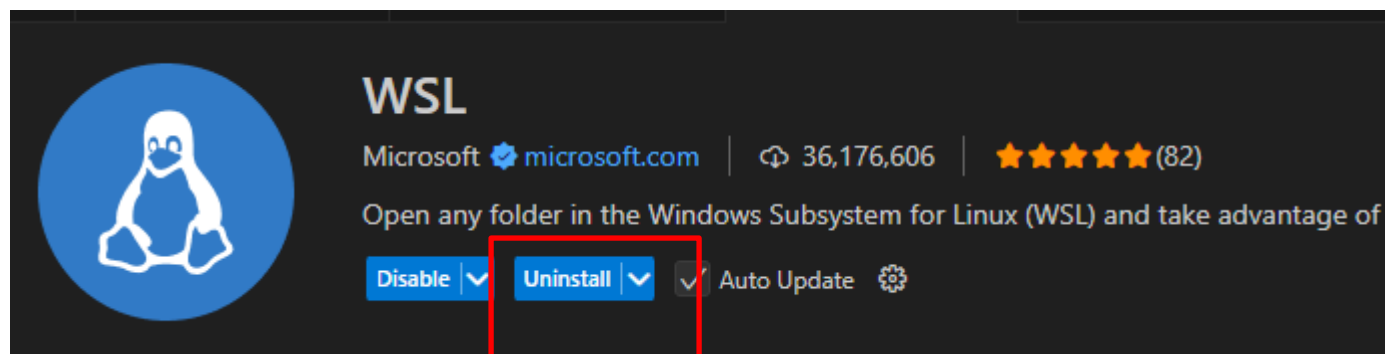
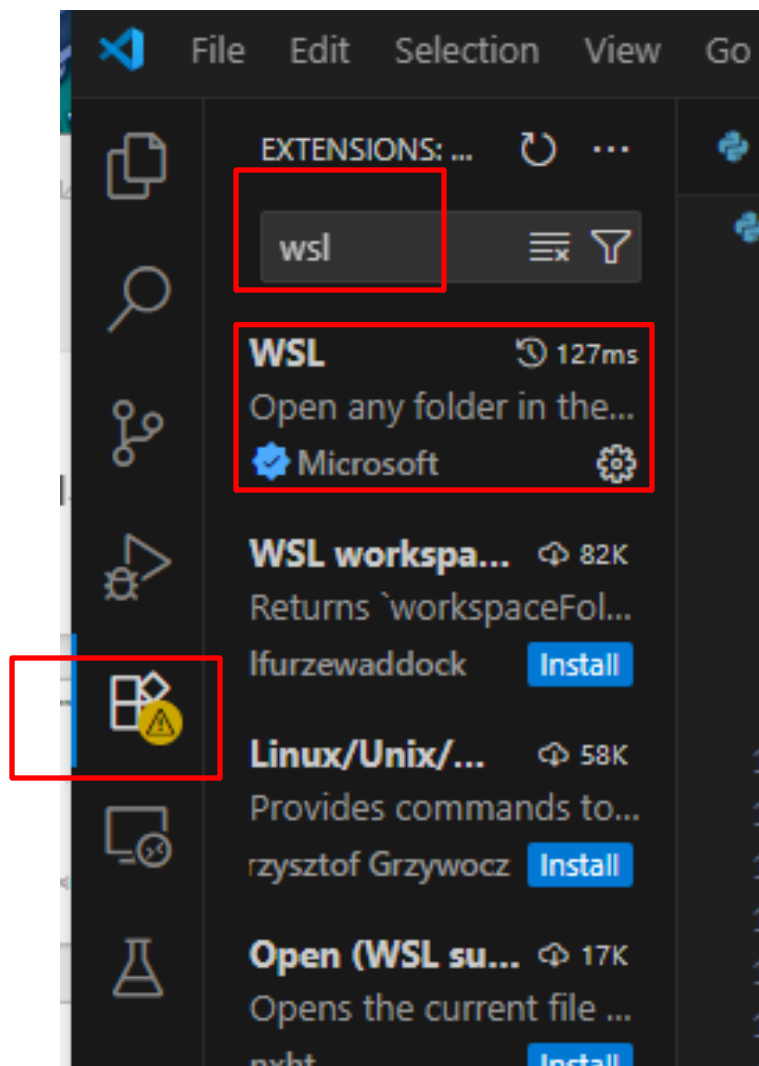


Visual Studio Code Docs Updates Blog API

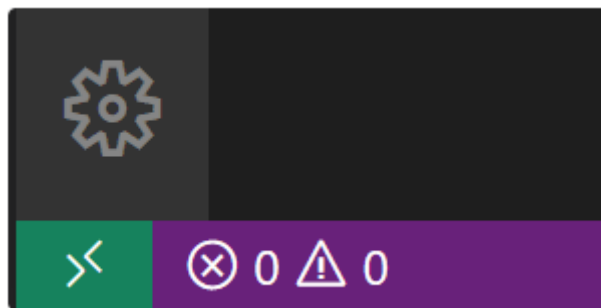


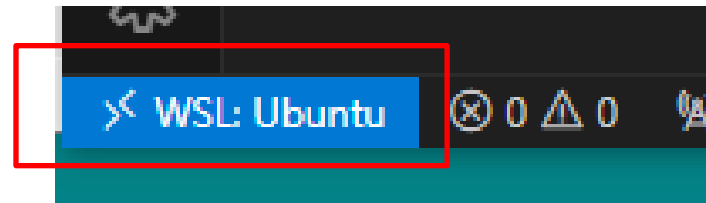
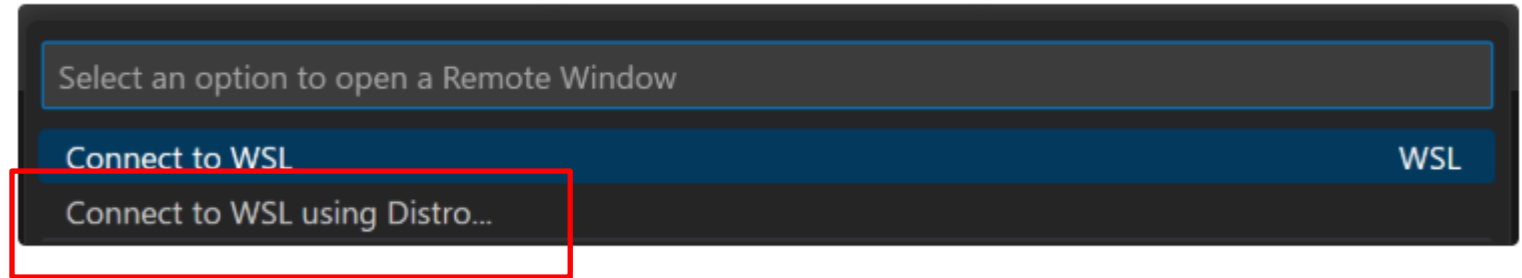
User Installer	x64	Arm64
System Installer	x64	Arm64
.zip	x64	Arm64
CLI	x64	Arm64



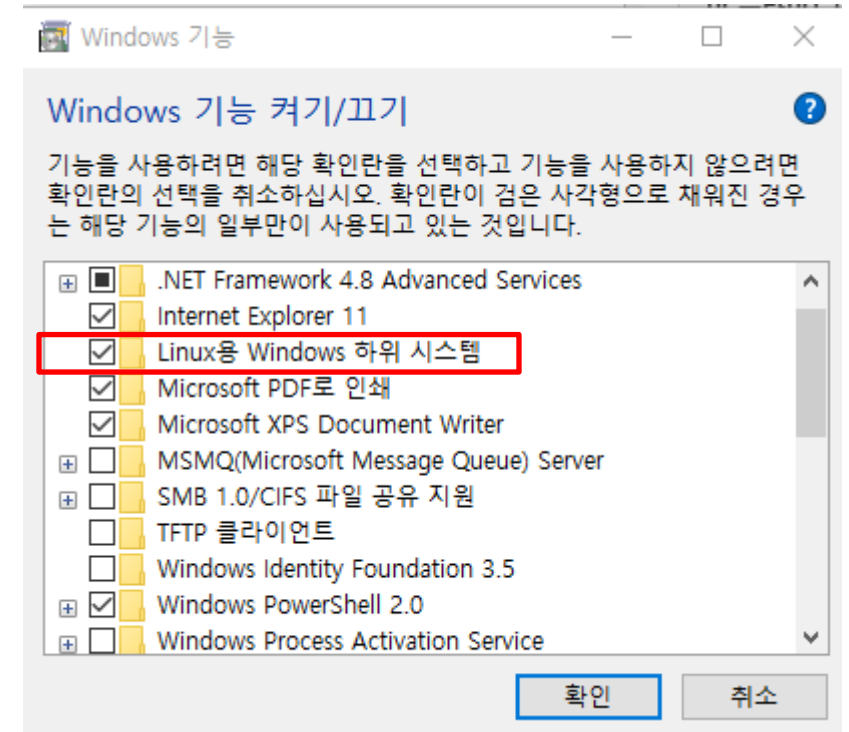


Uninstall 이라고 나오면 이미 설치된 것
최신 버전에는 포함되어 있음





- VSCode 하단에 아이콘 선택하면 상단에 우측 처럼 나타남
- Connect to WSL using Distro 선택하고 ubuntu 선택 → wsl 연결됨
- 연결이 안될 경우 Windows search bar에 윈도우라고 치면 Windows 기능 켜기 끄기 실행 후 Linux용 Windows 하위 시스템 선택 후 확인



GCC 컴파일러 설치하기

- VS Code 메뉴에서 view 선택 후 Terminal 선택 → 아래와 같은 창 나타남
- 아래 명령어 입력
 - sudo apt update
 - sudo apt install build-essential
- Wsl 설치 시 만든 패스워드 입력
- 컴파일러 설치 시작

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

npp@DESKTOP-TQ836S2:~\$ sudo apt update
sudo apt install build-essential
[sudo] password for npp:

이전에 설정한 패스워드 입력

- VS Code 메뉴에서 New File 선택 → EchoServer.c 입력 후 엔터 → 기본 폴더에 파일 생성
- 서버 코드 입력

```
home > npp > C EchoServer.c
```

```
1  Generate code (Ctrl+I), or
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

bash + ▾ □ □ ...

```
npp@DESKTOP-TQ836S2:~$ sudo apt update
```

```
sudo apt install build-essential
```

```
Setting up libheif1:amd64 (1.17.6-1ubuntu4.1) ...
```

```
Setting up libgd3:amd64 (2.3.3-9ubuntu5) ...
```

```
Setting up libc-devtools (2.39-0ubuntu8.5) ...
```

```
Setting up libheif-plugin-aomdec:amd64 (1.17.6-1ubuntu4.1) ...
```

```
Setting up libheif-plugin-libde265:amd64 (1.17.6-1ubuntu4.1) ...
```

```
Setting up libheif-plugin-aomenc:amd64 (1.17.6-1ubuntu4.1) ...
```

```
Processing triggers for libc-bin (2.39-0ubuntu8.5) ...
```

```
Processing triggers for man-db (2.12.0-4build2) ...
```

```
npp@DESKTOP-TQ836S2:~$ ls
```

```
EchoServer.c
```

```
npp@DESKTOP-TQ836S2:~$
```

Ln 12, Col 1 Spaces: 4 UTF-8 LF {} C

- 프로그램 코드 저장 후 컴파일

```
npp@DESKTOP-TQ836S2:~$ gcc -o server EchoServer.c
```

- 에러 없음을 확인한 후 실행

```
npp@DESKTOP-TQ836S2:~$ ./server
usage : ./server [port]
npp@DESKTOP-TQ836S2:~$ ./server 5000
Server : wating connection request.
^C
```

- VS Code 메뉴에서 New File 선택 → EchoClient.c 입력 후 엔터 → 기본 폴더에 파일 생성
- 클라이언트 코드 입력 후 저장
- 컴파일

서버 실행하기

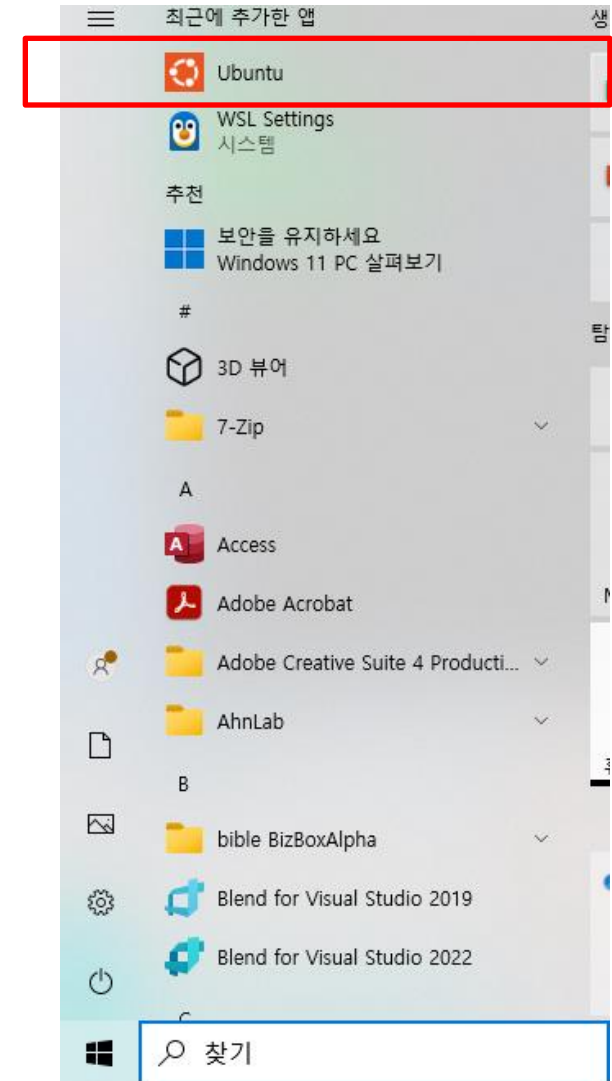
./server 5000

```
npp@DESKTOP-TQ836S2:~$ ./server 5000  
Server : wating connection request.
```

클라이언트 실행하기

- Ubuntu 실행한 후
./client 127.0.0.1 5000

```
npp@DESKTOP-TQ836S2: ~  
npp@DESKTOP-TQ836S2:~$ ls  
EchoClient.c EchoServer.c client server  
npp@DESKTOP-TQ836S2:~$ ./client 127.0.0.1 5000  
Message from server: Hello, I am a client  
npp@DESKTOP-TQ836S2:~$
```



실행 결과

서버는 계속 실행하고 있기 때문에 클라이언트를 반복 실행하면 동일한 결과를 계속 얻을 수 있음

```
npp@DESKTOP-TQ836S2:~$ ./server 5000
Server : wating connection request.
Server : 127.0.0.1 client connected.
Server : 127.0.0.1 client closed.
█
```

```
npp@DESKTOP-TQ836S2:~$ ./client 127.0.0.1 5000
Message from server: Hello, I am a client
npp@DESKTOP-TQ836S2:~$
```

VSCode와 wsl 연결을 끊으려면 File > Close remote connection 으로 연결 종료하면
다른 환경 사용 가능